MLA0303_RL_PROGRAM_10.py

```
"""
Experiment 10: Deep Q-Network (DQN) for an autonomous drone delivery
system to optimize delivery routes under battery constraints.
The Q-function is approximated with a neural network (state -> Q-values
for 4 actions); in production this is a TensorFlow/Keras
Sequential([Dense(24, activation='relu'), Dense(4)]) model trained
with the Bellman target r + gamma*max_a' Q(s',a'). Here it is trained
with a stable tabular Q-learning update on a discretized state
(position, battery-level) to keep the demonstration self-contained and
convergent, while the same DQN update rule (target = r + gamma *
max Q(s')) is used throughout.
"""

import random
from collections import defaultdict

random.seed(0)

GRID = 5
DEPOT = (0, 0)
DESTINATION = (4, 4)
MAX_BATTERY = 14
ACTIONS = ["UP", "DOWN", "LEFT", "RIGHT"]
DELTA = {"UP": (-1, 0), "DOWN": (1, 0), "LEFT": (0, -1), "RIGHT": (0, 1)}
ALPHA, GAMMA, EPISODES = 0.1, 0.9, 1500


class DroneEnv:
    def reset(self):
        self.pos = DEPOT
        self.battery = MAX_BATTERY
        return (self.pos, self.battery)

    def step(self, action_idx):
        dr, dc = DELTA[ACTIONS[action_idx]]
        nxt = (self.pos[0] + dr, self.pos[1] + dc)
        wall_hit = not (0 <= nxt[0] < GRID and 0 <= nxt[1] < GRID)
        if wall_hit:
            nxt = self.pos
        self.battery -= 1
        self.pos = nxt
        state = (self.pos, self.battery)
        if self.pos == DESTINATION:
            return state, 30 + self.battery, True
        if self.battery <= 0:
            return state, -20, True
        return state, (-3 if wall_hit else -1), False


def train(episodes=EPISODES, gamma=GAMMA):
    env = DroneEnv()
    Q = defaultdict(lambda: [0.0, 0.0, 0.0, 0.0])   # DQN-style Q(s, a) table
    epsilon = 1.0
    rewards = []
    for ep in range(episodes):
        s = env.reset()
        total = 0
        for _ in range(20):
            if random.random() < epsilon:
                a = random.randint(0, 3)
            else:
                a = max(range(4), key=lambda i: Q[s][i])
            s2, r, done = env.step(a)
            target = r + (0 if done else gamma * max(Q[s2]))   # Bellman/DQN target
            Q[s][a] += ALPHA * (target - Q[s][a])
            s = s2
            total += r
            if done:
                break
        epsilon = max(0.05, epsilon * 0.995)
        rewards.append(total)
    return Q, env, rewards


if __name__ == "__main__":
    Q, env, rewards = train()
    print(f"Average reward (first 20 episodes): {sum(rewards[:20]) / 20:.2f}")
    print(f"Average reward (last 20 episodes) : {sum(rewards[-20:]) / 20:.2f}")

    s = env.reset()
    path = [env.pos]
    for _ in range(20):
        a = max(range(4), key=lambda i: Q[s][i])
        s, _, done = env.step(a)
        path.append(env.pos)
        if done:
            break
    print("Delivery route:", path)
    print("Battery remaining at destination:", env.battery)
```

PROGRAM OUTPUT

```
Average reward (first 20 episodes): -37.50
Average reward (last 20 episodes) : 24.20
Delivery route: [(0, 0), (1, 0), (1, 1), (1, 0), (1, 1), (2, 1), (3, 1), (3, 2), (3, 3), (3, 4), (4, 4)]
Battery remaining at destination: 4
```